BS TECHNOTRONICS PVT LTD • ENTERPRISE IOT & TELEMATICS PLATFORM

BSTPL Customer Vault System

Complete Codebase & Runtime Execution Documentation Manual

Exhaustive Architectural Analysis Across All Backend (FastAPI, TiDB/SQLite, ORM, Dynamic Tables) and Frontend (Vanilla JS, SheetJS, HTML5 Modals, Real-time Grids) Modules.

System Scope & Architectural Blueprint:
The **BSTPL Customer Vault** is an enterprise telematics inventory and customer asset vault engineered to process, validate, and monitor tens of thousands of IoT SIM cards and GPS tracking devices across telecommunications providers (Sensorise, Airtel, Vodafone, Jio).
Unlike traditional monolithic database designs, the system incorporates a **Dual-Write Dynamic Isolation Architecture**: a unified master table (sim_devices) serves global search, auditing, and bulk operations, while **each customer organization is dynamically provisioned its own isolated physical SQL table** (e.g. c7_sharon_logistics) with automatic schema reflection and real-time synchronization.
The frontend is built with high-performance responsive Vanilla JavaScript and native HTML5 dialogs, providing sub-millisecond client validations (including strict 15-digit IMEI and 19/20-digit ICCID integrity checks) and instant error self-healing consoles.
This publication-grade document explains **why every core line was authored** and **how it executes at runtime** across both frontend and backend tiers.

1. Complete Repository Directory & File Catalog

Comprehensive inventory of all functional modules across backend services, client-side interfaces, and integration subsystems:

File Path	Language	Size	Core Architectural Role & System Purpose
backend/app.py	Python	95 lines	Server entrypoint, socket port availability checker, cloud/local host auto-detection, Uvicorn launcher.
backend/main.py	Python	499 lines	FastAPI core app instance, CORS middleware, router registrations, static asset mounting, startup lifecycle.
backend/database.py	Python	176 lines	SQLAlchemy connection pooling, TiDB Cloud SSL verification, local SQLite WAL fallback, get_db() session provider.
backend/models.py	Python	108 lines	Declarative ORM entity classes: User, Customer, Device (sim_devices), ColumnRegistry, ActivityLog, ErroredRecord.
backend/schemas.py	Python	529 lines	Pydantic validation schemas for API inputs, sanitization, response models, and user role permission specs.
backend/auth.py	Python	386 lines	Bcrypt password hashing, JWT token encoding/decoding, token invalidation versions, RBAC security dependencies.
backend/customer_tables.py	Python	308 lines	Dynamic per-customer SQL table engine (c<id>_<name>), table discovery, DDL migration, dialect-aware bulk upsert.
backend/routes_customers.py	Python	254 lines	Customer CRUD, safe recycling bin (/bin), restore, customer renaming, duplicate name validation, table synchronization.
backend/routes_data.py	Python	4041 lines	Master device querying, multi-column search, pagination, bulk CSV/Excel ingestion, exact length validation, error bin.
backend/routes_activity.py	Python	164 lines	Audit trail API endpoints, user action history retrieval, IP logging.
backend/routes_users.py	Python	343 lines	User administration, role assignment (Admin, Standard, Limited), password management.
backend/sensorise/*	Python	4800+ lines	External telematics gateway: Sensorise IoT SIM API client, OAuth token management, background synchronization.
frontend/inventory.html	HTML/JS	8777 lines	Unified master inventory grid, live filtering, customer summary view toggle, batch export, modal management.
frontend/sim-upload.html	HTML/JS	6921 lines	Drag-and-drop Excel upload, client SheetJS parsing, exact-length verification (15 IMEI, 19/20 ICCID), rectification modal.
frontend/fetch-data.html	HTML/JS	5525 lines	Sensorise IoT sync console, real-time sync triggering, telemetry audit logs, delta detection view.
frontend/settings.html	HTML/JS	5306 lines	Dynamic custom column registry, user administration panel, API token generator, appearance settings.
frontend/edit-delete.html	HTML/JS	4276 lines	Batch device operations, bulk customer reassignment, safe deletion to recycling bin, batch restoration.
frontend/errors.html	HTML/JS	3886 lines	Quarantine container review console, inline errored row editing, re-upload into active inventory, move to cart.
frontend/dashboard.html	HTML/JS	3714 lines	Executive telemetry dashboard, KPI stat cards, dynamic chart visualizations, recent activity stream.

File Path	Language	Size	Core Architectural Role & System Purpose
frontend/alerts.html	HTML/JS	3695 lines	Device warranty and SIM expiry monitoring console, threshold filtering (7/15/30 days), per-customer breakdown.
frontend/cart.html	HTML/JS	2726 lines	Staged device management cart, bulk move, checkout confirmation, customer reassignment tray.
frontend/login.html	HTML/JS	1218 lines	Authentication gateway, credentials verification, JWT storage in localStorage, security session redirect.
frontend/cv-popup.js	JavaScript	298 lines	Universal modal manager (openModal, closeModal, keyboard escape, outside click dismiss), toast notification engine.
frontend/cart-modal.js	JavaScript	609 lines	Cart state manager, badge count sync, floating staging tray DOM injection, customer assignment listener.
frontend/app.js	JavaScript	149 lines	Global client setup, theme provider, authorization bearer interceptor, fetch wrappers.
frontend/cv-popup.css	CSS	427 lines	Design system styling for modern modals, glassmorphism cards, status badges, and animated toast popups.

2. Backend Core Modules — Code & Runtime Execution

Detailed line-by-line and statement block breakdown of server entrypoints, database connection resiliency, ORM data models, dynamic customer table engine, and RESTful routing services.

2.1 Server Entry Point & Port Resolution (backend/app.py)

Line / Target	Exact Code / Construct	Why Written (Architectural Purpose)	How It Executes (Runtime Mechanics)

Lines 16–19	`backend_dir = Path(__file__).resolve().parent`
	`if str(backend_dir) not in sys.path:`
	`sys.path.insert(0, str(backend_dir))`
		Ensures module import resolution succeeds regardless of the working directory from which Python was invoked (e.g. root vs backend).	Executes at module load time. Resolves absolute directory path of `backend/` and prepends it to Python's `sys.path` lookup list, enabling direct imports like `import models`.

Lines 29–37	`def find_available_port(host, starting_port, max_attempts=20):`
	`for p in range(starting_port, starting_port + max_attempts):`
	`with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:`
	`try:`
	`s.bind((host, p))`
	`return p`
	`except OSError:`
	`continue`
	`return starting_port`
		Prevents startup failure when default port 8000 is occupied by zombie processes, Docker, or previous server instances on Windows/Linux.	Opens an AF_INET TCP socket and tests `s.bind()`. If OS raises `[Errno 98] / [WinError 10013] Address in use`, catches exception and increments port; closes socket immediately on return.

Lines 54–68	`is_cloud = bool(os.environ.get('PORT') or os.environ.get('RENDER'))`
	`host = os.environ.get('HOST', '0.0.0.0' if is_cloud else '127.0.0.1')`
	`if env_port:`
	`port = int(env_port)`
	`else:`
	`port = find_available_port(host, 8000)`
		Adapts server binding dynamically between local safe loopback (`127.0.0.1`) and cloud container public interfaces (`0.0.0.0`).	Inspects OS environment variables. On Render/Railway/Heroku, binds to `$PORT`. Locally, checks availability starting at 8000 and prints user-friendly switch notifications.

Lines 71–91	`uvicorn.run('main:app', host=host, port=port, reload=True, reload_dirs=[str(backend_dir)], reload_excludes=['*.log', '*.db*', 'logs/*'])`	Launches the high-concurrency ASGI server with live code reloading while excluding transient log/database files from triggering infinite restart loops.	Invokes Uvicorn ASGI event loop. Sets file watchers on backend directory. When files change, restarts workers unless matched by `reload_excludes`.

2.2 Database Layer & Connection Resilience (backend/database.py)

Line / Target	Exact Code / Construct	Why Written (Architectural Purpose)	How It Executes (Runtime Mechanics)

Lines 22–26	`env_file = Path(__file__).parent / '.env'`
	`if env_file.exists():`
	`load_dotenv(env_file, override=False)`
	`else:`
	`load_dotenv(override=False)`
		Loads secrets and connection strings from `.env` while preserving production environment variables injected by host containers.	Parses key-value pairs from disk into `os.environ`. `override=False` ensures system environment variables take precedence over file defaults.

Lines 35–51	`if DATABASE_URL:`
	`if DATABASE_URL.startswith('mysql://'):`
	`DATABASE_URL = DATABASE_URL.replace('mysql://', 'mysql+pymysql://', 1)`
	`else:`
	`DATABASE_URL = f'mysql+pymysql://{quote_plus(DB_USER)}:{quote_plus(DB_PASSWORD)}@{DB_HOST}:{DB_PORT}/{DB_NAME}'`
		Constructs standard SQLAlchemy URI using the PyMySQL driver, properly URL-encoding special characters in database credentials.	Validates prefix. If using individual variables, invokes `urllib.parse.quote_plus` on passwords (handling characters like '@', ':', '/') to construct connection URL.

Lines 62–93	`def _build_sqlite_engine(db_path):`
	`eng = create_engine(f'sqlite:///{db_path}', connect_args={'check_same_thread': False, 'timeout': 30})`
	`@event.listens_for(eng, 'connect')`
	`def set_sqlite_pragma(conn, _):`
	`cursor.execute('PRAGMA journal_mode=WAL')`
	`cursor.execute('PRAGMA busy_timeout=30000')`
		Configures resilient local SQLite fallback engine with Write-Ahead Logging (WAL) and custom SQL functions mimicking MySQL.	On connection, executes SQLite PRAGMAs to allow concurrent readers/writers without locking. Injects custom `date_format` and `left` Python callables into SQLite runtime.

Lines 105–150	`def _create_robust_engine():`
	`connect_args = {'ssl': {'ca': certifi.where()}}`
	`eng = create_engine(DATABASE_URL, pool_pre_ping=True, pool_recycle=240, pool_size=5)`
	`try:`
	`conn = eng.connect()`
	`conn.execute(text('SELECT 1'))`
	`except Exception:`
	`return _build_sqlite_engine(sqlite_path)`
		Provides zero-downtime resilience: connects to TiDB Cloud with SSL; if offline, network blocked, or DB down, transparently fails over to SQLite.	Initializes pool with `pool_pre_ping` (eliminates stale connection errors). Issues test query `SELECT 1`. On failure, prints alert and switches to local `customervault.db`.

Lines 159–176	`SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)`
	`def get_db():`
	`db = SessionLocal()`
	`try:`
	`yield db`
	`finally:`
	`db.close()`
		Provides thread-isolated database sessions to FastAPI route handlers via dependency injection with guaranteed cleanup.	Instantiates new session upon incoming request. Yields session to endpoint handler. In `finally` block, closes session and returns socket connection back to pool.

2.3 Relational ORM Entity Models (backend/models.py)

Line / Target	Exact Code / Construct	Why Written (Architectural Purpose)	How It Executes (Runtime Mechanics)

Lines 6–19	`class User(Base):`
	`__tablename__ = 'users'`
	`id = Column(Integer, primary_key=True)`
	`user_id = Column(String(30), unique=True)`
	`password_hash = Column(String(255))`
	`role = Column(String(10), default='limited')`
	`token_version = Column(Integer, default=0)`
		Defines authentication entity with role-based permissions and instant cross-device session revocation via `token_version`.	Maps to SQL table `users`. When user logs in, `password_hash` is verified via bcrypt. Bumping `token_version` invalidates all issued JWTs immediately.

Lines 21–30	`class Customer(Base):`
	`__tablename__ = 'customers'`
	`id = Column(Integer, primary_key=True)`
	`name = Column(String(150), unique=True)`
	`avatar = Column(Text, nullable=True)`
	`deleted_at = Column(DateTime, nullable=True)`
	`deleted_by = Column(String(30), nullable=True)`
		Represents client organizations. Implements soft-deletion so deleted customers can be reviewed or restored from the bin without data loss.	When customer is deleted, `deleted_at` timestamp is set. Active queries filter `WHERE deleted_at IS NULL`. Restoring sets `deleted_at = NULL`.

Lines 32–63	`class Device(Base):`
	`__tablename__ = 'sim_devices'`
	`id = Column(Integer, primary_key=True)`
	`imei = Column(String(20), unique=True, index=True)`
	`iccid = Column(String(30), index=True)`
	`expiry_date = Column(Date, index=True)`
	`extra = Column(JSON, nullable=True)`
		Master hardware and SIM inventory table. Indexes critical search keys (IMEI, ICCID, Expiry) and stores arbitrary dynamic CSV columns in JSON.	Indexed on `imei` for O(1) lookups during bulk upserts. `extra` field captures non-standard client columns dynamically without breaking schema.

Lines 85–108	`class ErroredRecord(Base):`
	`__tablename__ = 'sim_errored_records'`
	`id = Column(Integer, primary_key=True)`
	`customer_name = Column(String(150))`
	`imei = Column(String(100))`
	`error_reason = Column(String(500))`
		Quarantine table storing rows rejected during bulk upload (e.g. invalid length, malformed date) for user review and inline correction.	Allows partial batch commits: valid rows enter `sim_devices`, while flawed rows are isolated here so users can fix them without re-uploading entire files.

2.4 Dynamic Customer Isolation Engine (backend/customer_tables.py)

Line / Target	Exact Code / Construct	Why Written (Architectural Purpose)	How It Executes (Runtime Mechanics)
Lines 29–85	`DEVICE_BASE_COLUMNS = ['customer_id', 'device_sr_no', 'imei', 'iccid', ...] CREATE_TABLE_MYSQL_TPL = '''CREATE TABLE IF NOT EXISTS `{table}` ...'''`	Defines standard column schemas and database-agnostic DDL templates for dynamic customer tables.	Instantiated in memory at module load. Provides consistent schema definitions for creating tables on TiDB (InnoDB, utf8mb4) and SQLite.
Lines 87–114	`def customer_table_name(cid: int, customer_name: str) -> str: slug = slugify_customer_name(customer_name) return f'c{cid}_{slug}' if slug else f'c{cid}'`	Generates deterministic, SQL-safe physical table names isolated per customer (e.g. `c7_sharon_logistics`).	Converts name to lowercase, strips accents, replaces spaces and punctuation with underscores, truncates to 40 chars, and prefixes with `c<cid>_`.
Lines 136–178	`def ensure_customer_table(conn, table_name: str, extra_cols: list = None): # executes CREATE TABLE IF NOT EXISTS # queries existing columns # executes ALTER TABLE ADD COLUMN for missing extra cols`	Auto-provisions customer table on first upload and dynamically adds new columns if CSV contains extra fields.	Issues `CREATE TABLE IF NOT EXISTS`. Then inspects `information_schema.columns` (or `PRAGMA table_info`). If dynamic columns (`x_...`) are missing, issues `ALTER TABLE` DDL.
Lines 180–232	`def bulk_upsert_customer_table(conn, table_name: str, rows: list, ...): if dialect == 'mysql': sql = 'INSERT INTO ... ON DUPLICATE KEY UPDATE ...' else: sql = 'INSERT INTO ... ON CONFLICT(imei) DO UPDATE ...'`	Performs high-performance idempotent batch upsert into the customer-dedicated table matching target DB dialect.	Constructs multi-row parameterized SQL statement. If an IMEI already exists, updates non-null attributes in-place without throwing duplicate key errors.
Lines 234–256	`def mirror_devices_to_customer_tables(conn, devices: list, customer_map: dict): # groups devices by customer_id # ensures customer table # executes bulk upsert per customer`	Dispatches uploaded records from unified master stream into respective customer-isolated tables in a single transaction.	Partitions device rows by `customer_id` using a hash dictionary. Iterates unique customers, ensures table exists, and flushes rows into each customer table.
Lines 258–294	`def rename_customer_table(conn, cid, old_name, new_name): def drop_customer_table(conn, cid, customer_name):`	Handles lifecycle events when customer is renamed, merged, or permanently deleted from the system.	Renames physical SQL table using `ALTER TABLE ... RENAME TO ...`. On permanent delete, issues `DROP TABLE IF EXISTS` to reclaim storage space.

2.5 Customer Lifecycle & Safe Deletion Router (backend/routes_customers.py)

Line / Target	Exact Code / Construct	Why Written (Architectural Purpose)	How It Executes (Runtime Mechanics)
Lines 38–46	`@router.get('/bin') def bin_customers(db: Session = Depends(get_db)): # Queries customers WHERE deleted_at IS NOT NULL`	Retrieves soft-deleted customers in recycling bin. Registered BEFORE `/{cid}` route to prevent route collision (422 Unprocessable Content).	Executes outer join query between `customers` and `sim_devices` where `deleted_at IS NOT NULL`, grouping by customer ID to return deleted item counts.
Lines 48–63	`@router.get('/{cid}') def get_customer(cid: int, db: Session = Depends(get_db)): c = db.query(Customer).filter(Customer.id == cid, Customer.deleted_at.is_(None)).first()`	Fetches single active customer profile with active device count and 30-day expiry counts.	Validates integer `cid`. Queries database for active record. Calculates devices with `expiry_date <= today + 30 days` and returns JSON summary.
Lines 105–150	`@router.delete('/{cid}') def delete_customer_to_bin(cid: int, db: Session = Depends(get_db)): # sets c.deleted_at = now() # sets Device.deleted_at = now() for all customer devices`	Implements safe two-tier customer deletion: moves customer and all associated devices into recycling bin without data destruction.	Begins transaction. Sets `deleted_at = func.now()` and `deleted_by = user.user_id` on Customer and all linked Devices. Commits transaction and logs audit entry.

2.6 Data Ingestion, Hardware Length Validation & Query Engine (backend/routes_data.py)

Line / Target	Exact Code / Construct	Why Written (Architectural Purpose)	How It Executes (Runtime Mechanics)
Lines 320–380	`@router.get('/devices')` `def list_devices(page=1, page_size=10, q='', column='all', expiry_filter='all', ...):`	Core paginated inventory search API supporting full-text search across all columns and date threshold filters.	Constructs dynamic SQLAlchemy query. Applies `ILIKE / LIKE` filters against selected column. Computes total count via `func.count()` and returns paginated slice via `OFFSET / LIMIT`.

Lines 2515–2565	`def validate_device_lengths(device):`
	`if len(device.imei) != 15:`
	`raise ValidationError('IMEI must be exactly 15 digits')`
	`if len(device.iccid) not in (19, 20):`
	`raise ValidationError('ICCID must be 19 or 20 digits')`
		Enforces strict telematics hardware standards: IMEI must be exactly 15 numeric digits; ICCID must be 19 or 20 digits.	Executes string length check during ingestion. If length deviates, intercepts record, adds rejection reason, and routes record to `sim_errored_records`.

Lines 3210–3245	`@router.delete('/errors/permanent')` `def delete_all_errors_permanently(db: Session = Depends(get_db)):`	Clears all quarantined errored entries. Registered before `/errors/{error_id}` to prevent route shadowing.	Issues `DELETE FROM sim_errored_records` within transaction, logging activity and returning cleared row count.

3. Frontend Architecture & Client-Side Execution

In-depth breakdown of the client-side JavaScript execution layer, DOM event listeners, modal managers, SheetJS parser, exact-length verification, and interactive consoles.

3.1 Universal Client Core (cv-popup.js, cart-modal.js, app.js)

Line / Target	Exact Code / Construct	Why Written (Architectural Purpose)	How It Executes (Runtime Mechanics)
cv-popup.js: L15–45	`function openModal(id) { const dlg = document.getElementById(id); if (dlg && typeof dlg.showModal === 'function') dlg.showModal(); else if (dlg) dlg.setAttribute('open', ''); }`	Standardizes modal dialog management across modern browsers using native HTML5 `<dialog>` API with polyfill fallback.	Checks for native `HTMLDialogElement.showModal()`. If present, renders modal with backdrop and traps focus; adds event listener for backdrop clicks and Escape key.
cv-popup.js: L50–85	`function toast(msg, type='info', duration=4000) { const t = document.createElement('div'); t.className = `cv-toast cv-toast-${type}`; t.innerHTML = msg; container.appendChild(t); }`	Centralized non-intrusive alert/notification engine providing feedback on network actions (success, error, warning).	Dynamically creates DOM element with CSS classes, sets timer for auto-dismissal, animates slide-in, and removes node on animation completion.
cart-modal.js: L30–75	`function updateCartBadge() { const items = JSON.parse(localStorage.getItem('cart_items') || '[]'); document.querySelectorAll('.cart-badge-count').forEach(el => { el.textContent = items.length; }); }`	Synchronizes staged cart item count across multiple tabs and pages using localStorage state.	Reads serialized JSON array from `localStorage`. Updates all badge counters in DOM. Displays or hides the floating tray based on `items.length > 0`.
app.js: L12–38	`function authFetch(url, options = {}) { const token = localStorage.getItem('token'); if (!token && !url.includes('/login')) window.location.href = '/login.html'; options.headers = {...options.headers, 'Authorization': `Bearer ${token}`}; return fetch(url, options); }`	Global authenticated HTTP client interceptor that attaches JWT Bearer tokens and handles automatic session expiry redirects.	Intercepts every API fetch. Injects HTTP Authorization header. If response is 401 Unauthorized, clears token from localStorage and redirects user to login screen.

3.2 Authentication Gateway (frontend/login.html)

Line / Target	Exact Code / Construct	Why Written (Architectural Purpose)	How It Executes (Runtime Mechanics)
Lines 450–485	`form.addEventListener('submit', async (e) => { e.preventDefault(); const res = await fetch('/api/auth/login', { method: 'POST', body: JSON.stringify({ user_id, password }) }); })`	Intercepts default HTML form submission to perform asynchronous AJAX authentication without page reload.	Prevents default GET/POST page navigation. Collects inputs into JSON payload. Transmits POST to `/api/auth/login`. Displays loading spinner on submit button.
Lines 490–520	`if (res.ok) { const data = await res.json(); localStorage.setItem('token', data.access_token); localStorage.setItem('user', JSON.stringify(data.user)); window.location.href = '/home.html'; }`	Persists authenticated session token securely in client storage and navigates to main dashboard.	Decodes JSON response containing JWT token. Saves credentials to `localStorage`. Triggers browser redirection to `/home.html`.

3.3 Executive Telemetry Dashboard (frontend/dashboard.html)

Line / Target	Exact Code / Construct	Why Written (Architectural Purpose)	How It Executes (Runtime Mechanics)
Lines 120–165	`async function loadDashboardStats() { const [stats, act] = await Promise.all([ authFetch('/api/stats').then(r => r.json()), authFetch('/api/activity?limit=5').then(r => r.json()) ]); }`	Parallelizes asynchronous metrics loading on page initialization for optimal page load speed.	Dispatches concurrent network requests using `Promise.all`. Renders Total Devices, Active Customers, Expiring SIMs, and recent audit activity without UI freezing.
Lines 210–260	`new Chart(ctx, { type: 'doughnut', data: { labels: stats.providers, datasets: [{ data: stats.counts }] } });`	Visualizes inventory distribution across telematics providers (Airtel, Vodafone, Jio, Sensorise).	Instantiates HTML5 Canvas Chart.js instance. Maps provider categories to colored segments with responsive hover tooltips.

3.4 Unified Device Inventory Grid & Expiry Summary (frontend/inventory.html)

Line / Target	Exact Code / Construct	Why Written (Architectural Purpose)	How It Executes (Runtime Mechanics)
Lines 420–480	`function loadDevices(page=1) { const q = encodeURIComponent($('searchBox').value); const col = $('colFilter').value; const exp = $('expiryFilter').value; authFetch(`/api/devices?page=${page}&q=${q}&column=${col}&expiry_filter=${exp}`)... }`	Central device query dispatcher that coordinates multi-parameter filtering, search terms, and server-side pagination.	Builds parameterized query string. Sends GET request to `/api/devices`. Receives paginated device records, updates pagination controls, and triggers table DOM re-render.
Lines 1810–1860	`function renderCustomerExpirySummary(summary) { const container = $('customerExpiryContainer'); container.innerHTML = summary.map(c => `...`).join(''); }`	Renders dynamic per-customer inventory health cards showing total devices, active SIMs, and imminent expiry alerts.	Iterates aggregated customer statistics array. Generates accessible card components with color-coded status badges (green for active, red for expired).
Lines 2100–2150	`function switchExpiryView(mode) { if (mode === 'cards') { $('viewCards').style.display='block'; $('viewTable').style.display='none'; } else { $('viewCards').style.display='none'; $('viewTable').style.display='block'; } }`	Allows user to toggle between high-level visual card layout and detailed tabular breakdown without re-fetching data.	Toggles CSS `display` styles between card grid container and data table container in the DOM, maintaining clean single-table view.

3.5 Bulk Ingestion & Exact-Length Validation Pipeline (frontend/sim-upload.html)

Line / Target	Exact Code / Construct	Why Written (Architectural Purpose)	How It Executes (Runtime Mechanics)
Lines 2305–2415	`<dialog id='dlgImproperLengthAlert' class='dlg modal-card'> <span id='improperBadgeCount'>0</span> <button id='btnPushImproperFooter'>Push to Errored Entries</button> <button id='btnFocusRectifyTable'>Rectify Numbers Inline</button> </dialog>`	Declares dedicated interactive modal for intercepting and resolving records with improper IMEI or ICCID lengths before committing to database.	HTML5 modal containing two primary action cards: Option 1 to push directly to quarantine container (`sim_errored_records`), and Option 2 to expand the inline live-editing table.
Lines 4171–4230	`function findImproperLengthRows(rows) { const improper = []; rows.forEach((r, idx) => { const badImei = r.imei && r.imei.length !== 15; const badIccid = r.iccid && ![19, 20].includes(r.iccid.length); if (badImei || badIccid) improper.push({...r, rowIdx: idx}); }); return improper; }`	Client-side pre-validation filter that inspects every parsed row for strict telematics length compliance before network transmission.	Iterates in-memory parsed rows. Tests `imei.length === 15` and `iccid.length in [19, 20]`. Gathers violating rows into an array with row indices and specific violation reasons.
Lines 4233–4380	`async function promptImproperLengthModal(improperList, allRows) { openModal('dlgImproperLengthAlert'); $('improperBadgeCount').textContent = `${improperList.length} Records`; renderImproperTable(improperList); return new Promise(resolve => { ... }); }`	Orchestrates asynchronous user decision loop: halts upload pipeline until user chooses to quarantine or rectify records.	Populates modal counter badges. Generates live editable table. Binds Promise resolution to modal button clicks, returning staged indices to the upload loop.
Lines 4435–4510	`function handleSaveRectifiedInline() { // validates live inputs in DOM // updates parsedRows in place // pushes unrectified rows to errored entries closeModal('dlgImproperLengthAlert'); }`	Commits inline corrections made by the user in the modal table back to the active upload payload without losing the file state.	Reads edited values from table inputs. Re-verifies lengths. Validated rows are updated in `allRows`. Unrectified rows are marked for quarantine. Resumes upload stream.
Lines 4785–4840	`const improper = findImproperLengthRows(parsedRows); if (improper.length > 0) { const res = await promptImproperLengthModal(improper, parsedRows); stagedIndices = res.stagedIndices; } // proceed with valid and rectified rows`	Upload pipeline gatekeeper ensuring 100% data integrity before issuing batch POST request to backend server.	If improper rows exist, awaits modal resolution. Filters out quarantined indices. Sends remaining valid/rectified records in chunked batches to `/api/upload`.

3.6 Errored Entries Resolution Console (frontend/errors.html)

Line / Target	Exact Code / Construct	Why Written (Architectural Purpose)	How It Executes (Runtime Mechanics)
Lines 150–210	`async function loadErrors() { const res = await authFetch('/api/errors'); const errors = await res.json(); renderErrorTable(errors); }`	Fetches and displays all quarantined records that failed initial upload validation.	Issues GET to `/api/errors`. Renders error cards and table rows with highlighted error reason tags (e.g. 'Invalid IMEI Length: 14 digits').
Lines 320–370	`async function saveAndUploadCorrected(errorId, rowData) { const res = await authFetch(`/api/errors/${errorId}/upload`, { method: 'POST', body: JSON.stringify(rowData) }); }`	Allows operators to correct flawed records directly in the browser and immediately migrate them into the active customer inventory.	Sends rectified record to backend. Server validates schema, removes row from `sim_errored_records`, and inserts into `sim_devices` and customer-specific table.

3.7 Sensorise IoT Cloud Fetch & Sync Console (frontend/fetch-data.html)

Line / Target	Exact Code / Construct	Why Written (Architectural Purpose)	How It Executes (Runtime Mechanics)
Lines 180–230	`async function triggerSensoriseSync() { btn.disabled = true; btn.textContent = 'Syncing...'; const res = await authFetch('/api/sensorise/sync', { method: 'POST' }); pollSyncProgress(); }`	Initiates on-demand synchronization with external Sensorise IoT Cloud Gateway.	Dispatches background sync task on FastAPI backend. Initiates 2-second interval polling to stream live log updates and progress percentage to the UI console.

3.8 Warranty & SIM Expiry Alerts Engine (frontend/alerts.html)

Line / Target	Exact Code / Construct	Why Written (Architectural Purpose)	How It Executes (Runtime Mechanics)
Lines 210–270	`function filterByExpiry(days) { const filtered = allDevices.filter(d => { const diff = Math.ceil((new Date(d.expiry_date) - new Date()) / (86400000)); return diff <= days; renderAlertsGrid(filtered); }`	Filters hardware and SIM inventory by upcoming expiration horizon (7, 15, 30, or 60 days).	Computes day delta between device `expiry_date` and client system date. Filters in-memory device collection and updates alerts table with warning badges.

3.9 Batch Operations & Customer Reassignment (frontend/edit-delete.html)

Line / Target	Exact Code / Construct	Why Written (Architectural Purpose)	How It Executes (Runtime Mechanics)
Lines 310–360	`async function batchMoveCustomer(deviceIds, targetCustomerId) { const res = await authFetch('/api/devices/batch-reassign', { method: 'POST', body: JSON.stringify({ ids: deviceIds, customer_id: targetCustomerId }) }); }`	Enables bulk reassignment of devices between client organizations in a single atomic transaction.	Transmits array of selected device IDs and destination customer ID. Backend updates `sim_devices` and moves rows between customer dynamic tables.

3.10 System Settings & Custom Column Manager (frontend/settings.html)

Line / Target	Exact Code / Construct	Why Written (Architectural Purpose)	How It Executes (Runtime Mechanics)
Lines 140–190	`async function registerCustomColumn(colName) { const res = await authFetch('/api/columns', { method: 'POST', body: JSON.stringify({ name: colName }) }); }`	Dynamic schema management interface allowing administrators to register new custom telematics attributes without database migrations.	Sends new column name to `/api/columns`. Server adds column to `sim_column_registry` and updates dynamic customer table DDL templates.

4. End-to-End System Execution Lifecycle Traces

Comprehensive walkthroughs showing how requests propagate across Browser DOM, HTTP Network, FastAPI Middleware, Database Transactions, and TiDB/SQLite dynamic customer tables.

Trace 1: End-to-End Bulk SIM Ingestion & Exact-Length Validation Pipeline

Lifecycle Phase	System Action, Memory State, & Database Transaction
Step 1: File Selection & Drag-and-Drop	User selects or drags an Excel/CSV file onto `sim-upload.html`. Browser `FileReader` reads binary stream into memory.
Step 2: Client SheetJS Parsing	`XLSX.read(data, {type: 'binary'})` parses worksheets into structured JavaScript object array with column header normalization.
Step 3: Exact-Length Verification Gate	`findImproperLengthRows(parsedRows)` inspects every row: IMEI must be exactly 15 digits; ICCID must be 19 or 20 digits.
Step 4: Interception Modal Display	If violating rows are found, upload halts and `#dlgImproperLengthAlert` modal opens with Option 1 (Quarantine) and Option 2 (Rectify Inline).
Step 5: User Action Execution	User either pushes flawed rows directly to `sim_errored_records` or rectifies numbers in the inline table, dynamically updating badge counters.
Step 6: Network Transmission	Clean and rectified records are bundled into JSON batches and streamed to `POST /api/upload` with Bearer JWT token.
Step 7: Server Validation & Master Insert	FastAPI validates payload via `schemas.DeviceIn`. Inserts valid records into master `sim_devices` table inside database transaction.
Step 8: Dynamic Customer Table Mirroring	`customer_tables.mirror_devices_to_customer_tables()` resolves `c<id>_<name>` table, checks schema, and executes dialect-aware bulk upsert.
Step 9: Real-time UI Confirmation	Server returns batch summary JSON. Client displays success toast, updates cart badges, and redirects user to active inventory grid.

Trace 2: Error Quarantine, Review, & Self-Healing Lifecycle

Lifecycle Phase	System Action, Memory State, & Database Transaction
Step 1: Row Rejection & Quarantine	Any record failing database constraints, missing customer mapping, or having improper lengths is stored in `sim_errored_records`.
Step 2: Quarantine Console Inspection	Operator visits `errors.html`. The page queries `GET /api/errors` and renders records with explicit error reason tags.
Step 3: Inline Field Correction	Operator clicks 'Edit Row'. DOM inputs become editable; operator enters verified 15-digit IMEI or selects correct customer.
Step 4: Migration Re-submission	Operator clicks 'Save & Upload'. Client issues `POST /api/errors/{id}/upload` with corrected payload.
Step 5: Atomic Database Migration	Server begins transaction: inserts rectified record into `sim_devices` and `c<id>_<slug>`, then deletes row from `sim_errored_records`.
Step 6: UI Reconciliation	DOM removes the error row with a smooth fade animation, increments active inventory counter, and displays confirmation toast.

Trace 3: Dynamic Customer Isolation & Schema Synchronization Lifecycle

Lifecycle Phase	System Action, Memory State, & Database Transaction
Step 1: Customer Creation / Discovery	A new customer is created or detected during upload. `customer_table_name(cid, name)` computes SQL identifier (e.g. `c12_acme_corp`).
Step 2: Table Existence Verification	`ensure_customer_table()` queries `information_schema.tables` (TiDB) or `sqlite_master` (SQLite) to check if table exists.
Step 3: DDL Table Creation	If missing, executes `CREATE TABLE IF NOT EXISTS` with InnoDB engine, utf8mb4 charset, and indexed primary key on `imei`.
Step 4: Schema Reflection & Migration	Checks if incoming data has custom CSV attributes. If missing from customer table, dynamically issues `ALTER TABLE ADD COLUMN`.
Step 5: High-Performance Bulk Upsert	Executes `INSERT INTO c<id>_<slug> VALUES (...) ON DUPLICATE KEY UPDATE` (TiDB) or `ON CONFLICT DO UPDATE` (SQLite).
Step 6: Customer Rename / Merge Handling	If customer name changes, `rename_customer_table()` issues `ALTER TABLE RENAME TO`. On customer deletion, table is preserved in bin.

Trace 4: Sensorise IoT Telematics Cloud Synchronization Lifecycle

Lifecycle Phase	System Action, Memory State, & Database Transaction
Step 1: Scheduled / Manual Trigger	Cron job or user click on `fetch-data.html` triggers synchronization via `POST /api/sensorise/sync`.
Step 2: Token Authentication Handshake	Backend checks cached Sensorise OAuth token. If expired, initiates authentication handshake using API credentials to acquire fresh token.
Step 3: Paginated Telemetry Fetching	Streams device telemetry pages from Sensorise REST API (ICCID, SIM status, activation date, expiry date, data usage).
Step 4: In-Memory Delta Computation	Compares fetched telematics records against local `sim_devices` records to identify status differences and expiry renewals.
Step 5: Bulk Database Synchronization	Updates device records in `sim_devices` and mirrors updated attributes to customer-specific tables.
Step 6: Audit Trail & Real-time Console Log	Logs sync outcome into `sim_activity_log`. Client console receives live event stream with updated inventory counts.